



دانشگاه دامغان

دانشکده فنی و مهندسی

پایان نامه دوره کارشناسی مهندسی کامپیوتر

سایت اشتراک گذاری API با پایتون

سجاد رضاقلی زاده

استاد راهنما:

دکتر زهره کریمی

تیر ۱۴۰۵

بسمه تعالی

تشکر و قدردانی

از استاد راهنما و همه افرادی که در مسیر انجام این پروژه راهنمایی و حمایت کردند، سپاسگزاری می‌شود.

چکیده

این گزارش پروژه IranAPI Hub را به عنوان یک پلتفرم فارسی برای اشتراک گذاری، جستجو و مصرف API بررسی می کند. پروژه از بک اند Django REST Framework و فرانت اند React استفاده می کند و تجربه ای شبیه API Marketplace ارائه می دهد: کاربر می تواند کاتالوگ را مرور کند، مستندات را بخواند، وارد حساب خود شود، مصرف و اشتراک را ببیند، API منتشر کند و از ابزارهایی مانند Caller و Studio استفاده کند.

در این گزارش، متن با تمرکز بر تجربه واقعی کاربر، معماری فنی و مسیرهای توسعه آینده نوشته شده است. منابع نظری و فنی از داکيومنت های رسمی مانند Django, Django, Python, REST Framework, React, Node.js, Docker, MongoDB, OpenAPI و RapidAPI انتخاب شده اند. آزمون های بک اند با دستور `python manage.py test api` اجرا شدند و نتیجه در فصل آزمون آمده است.

کلیدواژه ها: پلتفرم اشتراک گذاری API, Django REST Framework, React, احراز هویت, OpenAPI

فهرست مطالب

د	فهرست علائم و نشانه‌ها
ه	فهرست جدول‌ها
و	فهرست نمودارها، عکس‌ها و نقشه‌ها
۱	۱ مقدمه
۱	۱-۱ زمینه پروژه
۱	۱-۲ بیان مسئله
۲	۱-۳ اهداف
۲	۱-۴ دامنه و محدودیت‌ها
۲	۱-۵ ذی‌نفعان
۲	۱-۶ سازمان‌دهی گزارش
۳	۱-۷ جمع‌بندی فصل
۴	۲ فناوری‌ها و پیشینه
۴	۲-۱ Django, Python و Django REST Framework
۴	۲-۲ MongoDB و مدل داده
۵	۲-۳ React, TypeScript و Vite
۵	۲-۴ Node.js و ابزارهای فرانت‌اند
۵	۲-۵ Docker Compose و اجرای محلی
۵	۲-۶ OpenAPI و RapidAPI
۵	۲-۷ جمع‌بندی فصل
۷	۳ تحلیل نیازمندی‌ها
۷	۳-۱ نقش‌های کاربری
۸	۳-۲ نیازمندی‌های عملکردی و غیرفعال
۹	۳-۳ ورودی‌ها، خروجی‌ها و اعتبارسنجی
۹	۳-۴ امنیت

۳-۵	جمع‌بندی فصل	۱۰
۴	معماری و طراحی	۱۱
۴-۱	نمای کلی	۱۱
۴-۲	نمودار معماری	۱۱
۴-۳	مدل داده	۱۲
۴-۴	مرزهای خارجی	۱۳
۴-۵	جمع‌بندی فصل	۱۳
۵	پیاده‌سازی	۱۴
۵-۱	مسیرهای بک‌اند	۱۴
۵-۲	لایه Repository	۱۴
۵-۳	Serializer و قرارداد پاسخ	۱۵
۵-۴	فرانت‌اند	۱۵
۵-۵	جریان‌های اصلی پیاده‌سازی	۱۵
۵-۶	بخش‌های فنی	۱۶
۵-۷	نمونه کد	۱۷
۵-۸	جمع‌بندی فصل	۱۷
۶	آزمون و ارزیابی	۱۸
۶-۱	راهبرد آزمون	۱۸
۶-۲	نتایج اجرا	۱۸
۶-۳	جدول آزمون‌ها	۱۹
۶-۴	محدودیت‌های آزمون	۲۰
۶-۵	جمع‌بندی فصل	۲۰
۷	نتایج و بحث	۲۱
۷-۱	ویژگی‌های پیاده‌سازی شده	۲۱
۷-۲	نمونه خروجی API	۲۱
۷-۳	نقاط قوت	۲۲
۷-۴	چالش‌ها و بدهی فنی	۲۲
۷-۵	بحث	۲۲
۷-۶	جمع‌بندی فصل	۲۲
۸	نتیجه‌گیری و کارهای آینده	۲۳
۸-۱	جمع‌بندی کار انجام‌شده	۲۳
۸-۲	دستاوردهای اصلی	۲۳
۸-۳	محدودیت‌های فعلی	۲۴

۲۴	کارهای آینده	۸-۴
۲۴	جمع‌بندی فصل	۸-۵

۲۵	منابع
----	-------

الف راهنمای اجرا و شواهد تکمیلی

۱-الف اجرای محلی
۲-الف اجرای آزمون بک‌اند
۳-الف مسیرهای مهم

فهرست علائم و نشانه‌ها

نماد	شرح
API	رابط برنامه‌نویسی کاربردی
DRF	Django REST Framework
SPA	برنامه تک‌صفحه‌ای
RTL	راست‌به‌چپ
ORM	نگاشت شیء-رابطه

فهرست جداول

۳-۱	نیازمندی‌های اصلی پروژه	۸
۴-۱	گروه‌های داده اصلی	۱۲
۵-۱	جریان‌های اصلی در پیاده‌سازی	۱۵
۵-۲	بخش‌های فنی و نقش آن‌ها در پروژه	۱۶
۶-۱	خلاصه آزمون‌های اجراشده	۱۹
الف-۱	مسیرهای مهم برای بررسی دستی سامانه	

فهرست نمودارها، عکس‌ها و نقشه‌ها

۳-۱	نمای نقش‌ها و جریان‌های اصلی پروژه	۷
۴-۱	معماری سطح بالای پروژه	۱۲

فصل ۱

مقدمه

این فصل تصویر کلی پروژه را توضیح می‌دهد: چرا چنین سامانه‌ای لازم است، کاربر با آن چه می‌کند، و گزارش از چه زاویه‌ای به پیاده‌سازی نگاه می‌کند. متن تلاش می‌کند فنی باشد، اما خشک و بریده‌بریده نباشد؛ یعنی خواننده ابتدا حس محصول را بگیرد و بعد وارد جزئیات معماری شود.

1-1 زمینه پروژه

IranAPI Hub یک پلتفرم فارسی برای معرفی، جستجو، انتشار و مصرف API است. فضای پروژه به بازارچه‌های API نزدیک است؛ جایی که کاربر می‌تواند سرویس‌ها را ببیند، مستندات را بخواند، پلن‌ها را مقایسه کند و بعد از ورود، مصرف و دسترسی‌های خود را مدیریت کند. این نگاه با مدل رایج API Hub و بازارچه‌های API هم‌خوان است [۱۱].

از نظر فنی، پروژه روی ترکیب Python, Django REST Framework, React, Vite, MongoDB و Docker Compose شکل گرفته است [۱، ۲، ۳، ۴، ۶، ۹، ۸]. این ترکیب برای سامانه‌ای مناسب است که هم به یک API ساخت‌یافته نیاز دارد و هم به یک رابط کاربری سریع و قابل توسعه.

2-1 بیان مسئله

کاربر چنین سامانه‌ای معمولاً نمی‌خواهد فقط یک فهرست ساده از سرویس‌ها ببیند. او باید بتواند API‌ها را جستجو کند، دسته‌بندی‌ها را بررسی کند، مستندات هر Endpoint را بخواند، قیمت یا پلن را بسنجد و بعد از ورود، وضعیت مصرف، کلیدها، اشتراک و سازمان خود را مدیریت کند. توسعه‌دهنده هم نیاز دارد API جدید منتشر کند، نمونه درخواست بفرستد و جریان کاری خود را در Studio تنظیم کند.

مسئله اصلی، پراکندگی تجربه کاربر است. وقتی مستندات، پرداخت، احراز هویت، مصرف و ابزار تست در چند مسیر جدا باشند، کاربر برای رسیدن به نتیجه باید چند بار زمینه ذهنی خود را

عوض کند. این پروژه تلاش می کند همین اجزا را در یک تجربه منسجم جمع کند؛ تجربه ای که هم برای کاربر عمومی قابل فهم باشد و هم برای توسعه دهنده کاربرد عملی داشته باشد.

3-1 اهداف

اهداف اصلی پروژه عبارت اند از:

- ارائه کاتالوگ API با جستجو، دسته بندی، امتیاز، پلن و مستندات.
 - فراهم کردن ثبت نام، ورود، خروج و نشست کاربر با مدل رایج احراز هویت وب.
 - نمایش Dashboard توسعه دهنده شامل پروفایل، API Key، مصرف، سازمان و اشتراک.
 - پشتیبانی از انتشار API، اجرای نمونه درخواست در Caller و ساخت جریان در Studio.
 - ارائه قرارداد OpenAPI برای خوانایی بهتر مسیرها و امکان اتصال ابزارهای دیگر [۱۰].
- این اهداف فقط به ظاهر سامانه مربوط نیستند. کاتالوگ و مستندات بخش عمومی محصول را می سازند؛ ورود، اشتراک و مصرف بخش خصوصی را شکل می دهند؛ و OpenAPI کمک می کند قرارداد سرویس برای انسان و ابزار قابل خواندن باشد.

4-1 دامنه و محدودیت ها

دامنه گزارش، نسخه فعلی پروژه و قابلیت هایی است که در آن قابل بررسی اند: وب اپلیکیشن فارسی، API بک اند، اجرای محلی و جریان های اصلی کاربر. اجرای محلی با Docker Compose در همین راستا مطرح می شود، چون برای بالا آوردن چند سرویس کنار هم طراحی شده است [۸]. در کنار قابلیت ها، چند محدودیت هم باید روشن گفته شود. تنظیمات تولیدی هنوز جای کار دارد، بخشی از رفتارها برای محیط توسعه مناسب ترند، پرداخت واقعی و اتصال کامل به سرویس های بیرونی نیاز به سخت سازی دارد، و آزمون مرورگر باید در چرخه ارزیابی منظم تر شود. بنابراین گزارش، پروژه را به عنوان یک نمونه کامل و قابل توسعه بررسی می کند، نه یک محصول تولیدی تمام شده.

5-1 ذی نفعان

ذی نفعان اصلی پروژه شامل کاربر مهمان، توسعه دهنده احراز شده، مدیر سیستم، ارائه دهنده ورود اجتماعی، سرویس مقصد API و بازارهای بیرونی مانند RapidAPI هستند [۱۱]. هر کدام از این گروه ها انتظار متفاوتی دارند: مهمان به شناخت سریع سرویس نیاز دارد، توسعه دهنده به ابزار و داده خصوصی، مدیر به کنترل سامانه، و سرویس های بیرونی به قرارداد قابل اعتماد.

6-1 سازمان دهی گزارش

فصل دوم فناوری ها را معرفی می کند. فصل سوم نیازمندی ها را دسته بندی می کند. فصل چهارم معماری را توضیح می دهد. فصل پنجم به پیاده سازی می پردازد. فصل ششم آزمون ها را گزارش می کند. فصل هفتم نتایج و چالش ها را جمع می کند و فصل هشتم مسیر ادامه کار را نشان می دهد.

7-1 جمع‌بندی فصل

در این فصل مشخص شد که پروژه یک پلتفرم اشتراک‌گذاری و مصرف API است؛ نه فقط یک تمرین بک‌اند یا یک رابط کاربری ساده. همین ماهیت دوگانه باعث می‌شود گزارش هم به تجربه کاربر توجه کند و هم به قرارداد فنی بین فرانت‌اند، بک‌اند و داده.

فصل ۲

فناوری‌ها و پیشینه

این فصل فناوری‌های اصلی پروژه را معرفی می‌کند و توضیح می‌دهد هر کدام در این سامانه چه نقشی دارند. معیار انتخاب واژه‌ها هم ساده است: هر جا اصطلاح فارسی دقیق و جاافتاده وجود دارد از فارسی استفاده شده، و هر جا ترجمه باعث ابهام می‌شود، اصطلاح اصلی انگلیسی آمده است.

1-2 Django، Python و Django REST Framework

Python زبان اصلی بک‌اند است؛ زبانی که به خاطر خوانایی و اکوسیستم گسترده، برای توسعه وب و ساخت سرویس‌های داده‌محور زیاد استفاده می‌شود [۱]. روی این زبان، Django چارچوب اصلی وب را فراهم می‌کند: تنظیمات، مسیرها، امنیت پایه، مدیریت کاربران، پنل مدیریت و ابزارهای آزمون [۲].

Django REST Framework لایه API را روی Django کامل‌تر می‌کند. مفاهیمی مثل View، Serializer، اعتبارسنجی ورودی و پاسخ JSON کمک می‌کنند مسیرهای عمومی و خصوصی سامانه با قرارداد منظم‌تری نوشته شوند [۳]. در این پروژه، همین ترکیب برای ساخت کاتالوگ، حساب کاربری، اشتراک، مصرف و ابزارهای توسعه‌دهنده استفاده شده است.

2-2 MongoDB و مدل داده

برای داده عملیاتی، پروژه به مدل سندی نزدیک است؛ مدلی که با ساختارهایی مانند کاربر، نشست، API، پلن، مصرف و جریان Studio راحت‌تر کار می‌کند. MongoDB در داکيومنت رسمی خود همین رویکرد سندمحور را برای ذخیره داده‌های انعطاف‌پذیر توضیح می‌دهد [۹]. در متن گزارش، وقتی از لایه داده حرف زده می‌شود، منظور همین بخش است که داده‌های محصول را برای مسیرهای بک‌اند آماده می‌کند.

این مدل برای بازارچه API منطقی است، چون همه داده‌ها قالب یکسان ندارند. مستندات یک API، پلن‌های قیمت‌گذاری، رخدادهای مصرف و جریان‌های Studio هر کدام شکل خودشان را دارند. مدل سندی باعث می‌شود این تفاوت‌ها با انعطاف بیشتری نگهداری شوند.

3-2 React، TypeScript و Vite

فرانت‌اند با React ساخته شده است؛ کتابخانه‌ای که برای ساخت رابط‌های کاربری مبتنی بر کامپوننت استفاده می‌شود [۴]. TypeScript هم به کد سمت کاربر نوع‌دهی اضافه می‌کند تا خطاهای ساده زودتر دیده شوند و قرارداد بین بخش‌های مختلف روشن‌تر بماند [۵].

Vite ابزار توسعه و build فرانت‌اند است. داکيومنت رسمی Vite روی سرعت راه‌اندازی و تجربه توسعه سریع تاکید دارد [۶]. این ویژگی برای پروژه‌ای با صفحه‌های مختلف مانند کاتالوگ، جزئیات API، Dashboard، Caller و Studio مفید است، چون توسعه رابط کاربری را روان‌تر می‌کند.

4-2 Node.js و ابزارهای فرانت‌اند

اکوسیستم فرانت‌اند پروژه روی Node.js و ابزارهای بسته‌بندی جاوااسکریپت می‌چرخد. Node.js اجرای جاوااسکریپت خارج از مرورگر را فراهم می‌کند و پایه بسیاری از ابزارهای توسعه فرانت‌اند است [۷]. کتابخانه‌هایی مثل TanStack Query، Axios و مجموعه‌های کامپوننتی، کار ارتباط با بک‌اند، مدیریت وضعیت درخواست‌ها و ساخت فرم‌ها و جدول‌ها را ساده‌تر می‌کنند.

در گزارش، نام این ابزارها بدون ترجمه آمده است، چون ترجمه فارسی آن‌ها معمولاً دقیق نیست و ممکن است خواننده را از اصطلاح رایج دور کند.

5-2 Docker Compose و اجرای محلی

Docker Compose برای تعریف و اجرای چند سرویس کنار هم استفاده می‌شود [۸]. در این پروژه، همین فضا باعث می‌شود بک‌اند و فرانت‌اند در کنار هم بالا بیایند و مسیر ارتباطی مشخصی داشته باشند. این روش برای ارائه، آزمون دستی و توسعه محلی مناسب است، هرچند جای پیکربندی تولیدی را نمی‌گیرد.

6-2 OpenAPI و RapidAPI

OpenAPI برای توصیف قرارداد API به کار می‌رود؛ یعنی مسیرها، ورودی‌ها، خروجی‌ها و پاسخ‌ها را به شکلی می‌نویسد که هم انسان و هم ابزار بتوانند آن را بخوانند [۱۰]. RapidAPI نیز نمونه شناخته‌شده‌ای از فضای بازارچه API است و برای فهم تجربه کاربر در کشف و مصرف API مرجع خوبی به شمار می‌آید [۱۱].

7-2 جمع‌بندی فصل

پشته فنی پروژه با ماهیت آن هماهنگ است: Django REST Framework برای ساخت API، React برای تجربه کاربری، MongoDB برای داده‌های سندی، Docker Compose برای اجرای

محلی، و OpenAPI برای خوانایی قرارداد سرویس. این ترکیب، فضای گزارش را هم به سمت یک محصول واقعی اشتراک‌گذاری API می‌برد.

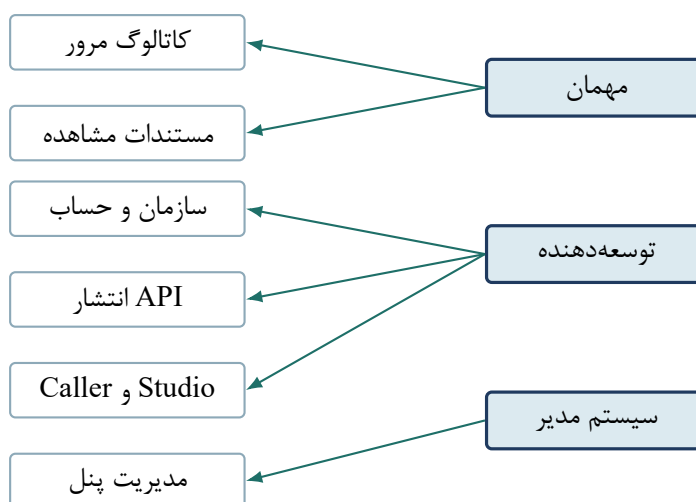
فصل ۳

تحلیل نیازمندی‌ها

این فصل نیازمندی‌های پروژه را از زاویه تجربه کاربر و رفتار سامانه دسته‌بندی می‌کند. هدف این نیست که فقط یک فهرست خشک ارائه شود؛ هر نیازمندی باید نشان دهد کاربر چه انتظاری دارد و سامانه برای پاسخ به آن چه رفتاری باید داشته باشد.

1-3 نقش‌های کاربری

سه نقش اصلی در پروژه دیده می‌شود: مهمان، توسعه‌دهنده احراز شده و مدیر سیستم. مهمان بخش عمومی کاتالوگ را می‌بیند. توسعه‌دهنده بعد از ورود، با Dashboard، انتشار API، Caller و Studio کار می‌کند. مدیر سیستم هم مسئول کنترل کلی و نگهداری داده‌هاست.



شکل 1-3 نمای نقش‌ها و جریان‌های اصلی پروژه

نقش مهمان بدون ورود می‌تواند بخش عمومی بازارچه را ببیند؛ این بخش برای جذب کاربر و معرفی API‌ها ضروری است. نقش توسعه‌دهنده پس از احراز هویت به امکاناتی دسترسی دارد که

با داده خصوصی کاربر ارتباط دارند، مانند کلید API، سازمان، اشتراک، مصرف، انتشار و ابزارهای آزمایشی. نقش مدیر سیستم بیشتر به نگهداری داده و پشتیبانی از مسیرهای مدیریتی مربوط می‌شود و از نظر سطح دسترسی از دو نقش دیگر حساس‌تر است.

2-3 نیازمندی‌های عملکردی و غیرفعال

جدول ۱-۳ نیازمندی‌ها را در دو گروه اصلی نشان می‌دهد. نیازمندی‌های عملکردی به کارهایی مربوط است که کاربر مستقیماً می‌بیند؛ مثل جستجوی API، ورود، انتشار یا مشاهده مصرف. نیازمندی‌های امنیتی و اجرایی کیفیت همین رفتارها را حفظ می‌کنند؛ مثل پنهان کردن کلیدها، پاسخ خطای قابل فهم و اجرای محلی با Docker Compose.

جدول 1-3 نیازمندی‌های اصلی پروژه

شناسه	نیازمندی	نوع	اولویت	وضعیت
R1	نمایش فهرست APIها با صفحه‌بندی، جستجو، دسته‌بندی و مرتب‌سازی	عملکردی	زیاد	انجام‌شده
R2	نمایش جزئیات، پلن‌ها، مستندات، endpointها و APIهای مشابه	عملکردی	زیاد	انجام‌شده
R3	ثبت‌نام، ورود، نشست، خروج و کشف ارائه‌دهندگان ورود اجتماعی	عملکردی	زیاد	انجام‌شده
R4	مدیریت کاربر، پروفایل، سازمان، اشتراک، پرداخت دستی و مصرف	عملکردی	زیاد	انجام‌شده
R5	انتشار API توسط کاربر احراز‌شده	عملکردی	متوسط	انجام‌شده
R6	اجرای درخواست نمونه در Caller و ثبت مصرف	عملکردی	متوسط	انجام‌شده
R7	استقرار جریان Studio و ثبت مصرف	عملکردی	متوسط	انجام‌شده
R8	بازگشت خطای یکنواخت برای اعتبارسنجی و دسترسی	غیرفعال	زیاد	انجام‌شده
R9	پنهان‌سازی کلیدها و توکن‌ها در پاسخ‌ها و لاگ‌ها	امنیتی	زیاد	انجام‌شده

شناسه	نیازمندی	نوع	اولویت	وضعیت
R10	اجرای محلی دو سرویس با استقرار Docker Compose	متوسط	مستند	

اولویت زیاد برای نیازهایی در نظر گرفته شده که نبود آن‌ها باعث از کار افتادن هسته محصول می‌شود؛ مانند کاتالوگ، احراز هویت، حساب کاربری و امنیت داده حساس. اولویت متوسط برای قابلیت‌هایی است که ارزش محصول را کامل‌تر می‌کنند، اما می‌توانند در نسخه‌های بعدی نیز تکامل پیدا کنند؛ مانند Studio، Caller و اجرای محلی.

3-3 ورودی‌ها، خروجی‌ها و اعتبارسنجی

ورودی‌های اصلی شامل مشخصات حساب، پروفایل، شناسه یا نامک API، فیلترهای جستجو، امتیاز، داده انتشار API، شناسه پلن اشتراک و گره‌های Studio است. در معماری مبتنی بر Django REST Framework، معمولاً Serializer مسئول کنترل و تبدیل همین داده‌هاست [۳].

خروجی‌های اصلی سامانه در قالب JSON بازگردانده می‌شوند و برای مصرف فرانت‌اند طراحی شده‌اند. پاسخ‌های عمومی معمولاً شامل داده کاتالوگ، مستندات، پلن‌ها و وضعیت سلامت هستند. پاسخ‌های خصوصی شامل داده‌هایی مانند پروفایل، سازمان‌ها، کلید پنهان‌شده، اشتراک و تاریخچه مصرف هستند. جداسازی این دو دسته خروجی اهمیت دارد، زیرا داده عمومی برای همه کاربران قابل نمایش است، اما داده خصوصی باید فقط بعد از احراز هویت برگردد.

اعتبارسنجی در این پروژه دو نقش دارد: نخست جلوگیری از ذخیره داده ناقص یا نامعتبر، و دوم تبدیل خطا به پاسخ قابل فهم برای Client. برای نمونه، هنگام ثبت امتیاز باید مقدار امتیاز معتبر باشد، هنگام انتشار API فیلدهای اصلی باید حاضر باشند و هنگام اجرای Studio ساختار گره‌ها باید قابل پردازش باشد.

4-3 امنیت

احراز هویت باید به شکلی باشد که کاربر بعد از ورود بتواند به بخش‌های خصوصی دسترسی داشته باشد، اما داده حساس او در پاسخ‌ها آشکار نشود. در پروژه‌هایی که با API Key، توکن و کوکی کار می‌کنند، این موضوع فقط یک انتخاب ظاهری نیست؛ بخشی از امنیت اصلی محصول است [۲، ۳].

از دید نیازمندی، امنیت فقط ورود کاربر نیست؛ بلکه کنترل دسترسی، نگهداری نشست، جلوگیری از افشای رازها و محدودکردن خروجی‌های حساس را نیز شامل می‌شود. آزمون‌های موجود نشان می‌دهند که پنهان‌سازی داده حساس در خروجی‌ها بررسی شده است. با این حال، برای محیط تولید باید رازهای پیکربندی از فایل تنظیمات خارج شوند و سیاست‌های سخت‌گیرانه‌تری برای کوکی، دامنه مجاز و خطاهای عملیاتی اعمال شود.

5-3 جمع‌بندی فصل

نیازهای پروژه حول کشف عمومی API، فضای کاری توسعه‌دهنده، مدیریت حساب و اشتراک، و امنیت نشست شکل گرفته‌اند. این نیازمندی‌ها مبنای فصل معماری هستند؛ چون لایه‌های سامانه باید همین نقش‌ها، جریان‌ها و محدودیت‌های امنیتی را پشتیبانی کنند.

فصل ۴

معماری و طراحی

این فصل معماری سامانه را با زبان ساده توضیح می‌دهد. مسیر کلی چنین است: کاربر از رابط React شروع می‌کند، درخواست به بک‌اند Django REST Framework می‌رسد، داده اعتبارسنجی می‌شود و در نهایت لایه داده نتیجه را آماده می‌کند.

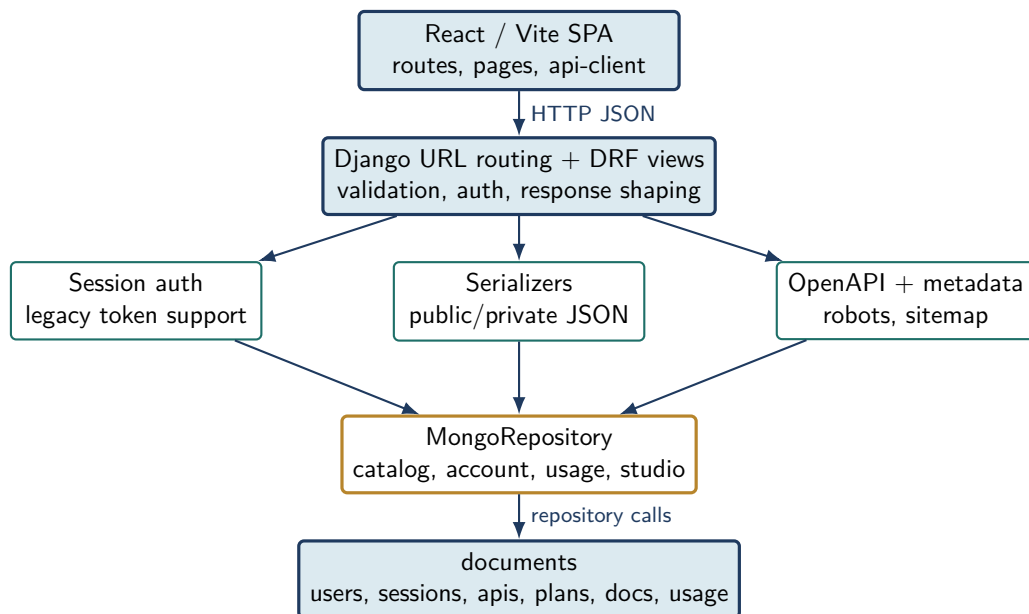
1-4 نمای کلی

معماری پروژه سه لایه اصلی دارد: رابط کاربری React، بک‌اند Django REST Framework و لایه داده. این مدل با الگوی رایج وب‌اپلیکیشن‌های مدرن هماهنگ است؛ فرانت‌اند تجربه کاربر را می‌سازد، بک‌اند قرارداد API را مدیریت می‌کند و لایه داده مسئول نگهداری اطلاعات محصول است [۴، ۳، ۹].

این طراحی باعث می‌شود کاربر نهایی بیشتر با یک برنامه تک‌صفحه‌ای روبه‌رو باشد، در حالی که عملیات داده از طریق API نسخه‌دار انجام می‌شود. نسخه‌دار بودن مسیرها برای نگهداری آینده مهم است، زیرا امکان تغییر یا اضافه کردن قراردادهای جدید را بدون شکستن Client‌های قدیمی فراهم می‌کند. همچنین نمایش پوسته SPA برای مسیرهای ناشناخته فرانت‌اند، با الگوی مسیریابی سمت Client سازگار است.

2-4 نمودار معماری

مطابق شکل ۴-۱، فرانت‌اند درخواست‌ها را به API بک‌اند می‌فرستد. بک‌اند، ورودی را اعتبارسنجی می‌کند، پاسخ را به قالب JSON درمی‌آورد و کار با داده را به لایه Repository می‌سپارد. در این معماری، هر لایه کار خودش را انجام می‌دهد. فرانت‌اند مسئول نمایش، فرم‌ها و حرکت کاربر بین صفحه‌هاست. بک‌اند مسئول احراز هویت، اعتبارسنجی و شکل پاسخ است. Repository هم جزئیات ذخیره‌سازی و بازیابی داده را پنهان می‌کند. همین جداسازی باعث می‌شود تغییر در ظاهر کاربر، کل منطق داده را به هم نریزد.



شکل 1-4 معماری سطح بالای پروژه

3-4 مدل داده

مدل داده پروژه حول چند گروه اصلی می‌چرخد: هویت کاربر، کاتالوگ API، دسترسی و مصرف، فضای کاری توسعه‌دهنده و اشتراک. این تقسیم‌بندی با مدل سندی MongoDB سازگار است، چون هر گروه می‌تواند ساختار و جزئیات خودش را داشته باشد [۹].

جدول 1-4 گروه‌های داده اصلی

گروه	مجموعه‌ها و مسئولیت
هویت	users, sessions و legacy_tokens برای کاربر، نشست و توکن سازگار
کاتالوگ	categories, apis, pricing_plans, documentations, api_endpoints و api_ratings
مصرف	access_grants و api_usage برای مجوز دسترسی و رخدادهای مصرف
فضای کاری توسعه‌دهنده	organizations و studio_flows برای امکانات

گروه	مجموعه‌ها و مسئولیت
اشتراک	subscription_plans, user_subscriptions و subscription_checkouts

گروه هویت پایه دسترسی کاربر را می‌سازد و به نشست‌ها و توکن‌های سازگار قدیمی متصل است. گروه کاتالوگ محتوای عمومی بازارچه را نگه می‌دارد؛ یعنی دسته‌ها، خود API‌ها، پلن قیمت‌گذاری، مستندات، نقاط انتهایی و امتیازها. گروه مصرف و اشتراک داده‌هایی را ثبت می‌کند که برای داشبورد، کنترل دسترسی و گزارش استفاده لازم‌اند. گروه فضای کاری نیز قابلیت‌های مخصوص توسعه‌دهنده مانند سازمان و جریان Studio را پشتیبانی می‌کند.

4-4 مرزهای خارجی

مرزهای خارجی پروژه شامل ارائه‌دهندگان ورود اجتماعی، سرویس مقصد API و بازارهای بیرونی مانند RapidAPI است [۱۱]. این مرزها باید روشن باشند، چون هر کدام سطحی از ریسک و وابستگی بیرونی وارد سامانه می‌کنند.

شناخت مرز خارجی برای تحلیل ریسک مهم است. ورود اجتماعی به پیکربندی ارائه‌دهنده وابسته است و در صورت نبود auth_url نمی‌تواند کامل اجرا شود. Caller و Studio نیز از نظر محصول به ارتباط با سرویس‌های بیرونی نزدیک هستند، اما در وضعیت فعلی بیشتر برای نمایش جریان و ثبت مصرف استفاده می‌شوند. بنابراین گزارش بین قابلیت پیاده‌سازی شده و اتصال تولیدی واقعی تفاوت می‌گذارد.

5-4 جمع‌بندی فصل

معماری پروژه یک SPA متصل به API یک‌پارچه است. این طراحی برای نمونه محصول مناسب است و فضای پروژه را به یک پلتفرم واقعی API نزدیک می‌کند. برای استفاده تولیدی، تصمیم نهایی درباره پایگاه داده، مدیریت رازها و اتصال‌های بیرونی باید دقیق‌تر شود.

فصل ۵

پیاده‌سازی

این فصل پیاده‌سازی را از زاویه رفتار سامانه توضیح می‌دهد. به جای تکیه روی جزئیات داخلی پروژه، مسیر ذهنی کاربر و نقش هر بخش فنی بیان می‌شود؛ چون خواننده گزارش باید بفهمد محصول چگونه کار می‌کند، نه این‌که درگیر نام داخلی فایل‌ها شود.

1-5 مسیرهای بک‌اند

بک‌اند مجموعه‌ای از مسیرهای نسخه‌دار دارد که بخش‌های اصلی محصول را پوشش می‌دهند: سلامت سامانه، قرارداد OpenAPI، ثبت‌نام، ورود، خروج، حساب کاربری، پروفایل، API Key، سازمان‌ها، اشتراک، مصرف، کاتالوگ، مستندات، Endpointها، Caller و Studio. استفاده از مسیرهای نسخه‌دار برای API کمک می‌کند توسعه آینده کنترل‌شده‌تر باشد و Clientها با تغییرات ناگهانی روبه‌رو نشوند [۳، ۱۰].

این مسیرها دو سطح تجربه می‌سازند. سطح اول عمومی است و برای مشاهده کاتالوگ، جزئیات API و مستندات استفاده می‌شود. سطح دوم خصوصی است و بعد از ورود کاربر فعال می‌شود؛ مثل Dashboard، مصرف، اشتراک، سازمان و انتشار API.

2-5 لایه Repository

لایه Repository منطق کار با داده را متمرکز می‌کند. ساخت کاربر، نگهداری نشست، چرخش API Key، خواندن کاتالوگ، ثبت مصرف Caller و ذخیره جریان‌های Studio از همین مسیر انجام می‌شود. این شیوه باعث می‌شود Viewها بیش از حد شلوغ نشوند و جزئیات داده در یک لایه مشخص بماند.

در پروژه‌ای شبیه بازارچه API، این جداسازی مهم است. داده‌های حساب، کاتالوگ، مستندات، اشتراک و مصرف شکل یکسانی ندارند. وقتی این بخش‌ها پشت یک Repository قرار می‌گیرند، تغییر در روش ذخیره‌سازی کمتر به بخش‌های دیگر نشت می‌کند.

3-5 Serializer و قرارداد پاسخ

Serializerها نقش مرز امن بین داده داخلی و خروجی قابل نمایش را دارند. داده‌ای که داخل سامانه نگهداری می‌شود، همیشه نباید با همان شکل به کاربر برگردد. برای نمونه، مقدار کامل API Key نباید در پاسخ عادی دیده شود و بهتر است فقط نسخه پنهان‌شده آن نمایش داده شود. این رویکرد با نقش Serializer در Django REST Framework هماهنگ است [۳].

در بخش انتشار API نیز Serializer کمک می‌کند داده ورودی قبل از ثبت بررسی شود: نام، آدرس پایه، روش احراز هویت، دسته و توضیح سرویس باید روشن باشند. نتیجه این کار، پاسخ‌هایی است که برای فرانت‌اند قابل پیش‌بینی‌ترند.

4-5 فرانت‌اند

فرانت‌اند تجربه اصلی کاربر را می‌سازد: صفحه مرور APIها، جزئیات سرویس، مستندات، قیمت‌گذاری، پرداخت، ورود، ثبت‌نام، Dashboard، سازمان‌ها، انتشار، Caller و Studio. معماری کامپوننتی React برای چنین تجربه‌ای مناسب است، چون بخش‌های تکرارشونده مثل فرم، کارت، جدول و وضعیت بارگذاری را می‌توان جدا نگه داشت [۴].

ارتباط فرانت‌اند با بک‌اند از طریق Clientهای مشخص انجام می‌شود. یک گروه درخواست‌ها برای کاتالوگ است، یک گروه برای احراز هویت، و یک گروه برای حساب کاربری و ابزارهای توسعه‌دهنده. این جداسازی باعث می‌شود صفحه‌ها کمتر به جزئیات آدرس‌ها وابسته باشند و نگهداری پروژه ساده‌تر شود.

5-5 جریان‌های اصلی پیاده‌سازی

جدول ۵-۱ مسیرهای اصلی محصول را از نگاه کاربر و سامانه نشان می‌دهد.

جدول 5-1 جریان‌های اصلی در پیاده‌سازی

جریان	بخش‌های درگیر	خروجی برای کاربر
مرور کاتالوگ	فرانت‌اند، Client، کاتالوگ، بک‌اند و لایه داده	فهرست قابل جستجو و صفحه‌بندی‌شده APIها
جزئیات API	صفحه جزئیات، مستندات، پلن‌ها و Endpointها	شناخت سرویس قبل از مصرف
ورود و نشست	فرم ورود، احراز هویت، کوکی، نشست و پاسخ کاربر	دسترسی به بخش خصوصی

جریان	بخش‌های درگیر	خروجی برای کاربر
Dashboard	حساب کاربری، مصرف، اشتراک، سازمان و API Key	مدیریت وضعیت توسعه‌دهنده
انتشار API	فرم انتشار، اعتبارسنجی و ثبت در کاتالوگ	اضافه‌شدن سرویس جدید به محصول
Studio و Caller	ابزار تست، جریان کاری و ثبت مصرف	آزمایش و طراحی جریان‌های مرتبط با API

6-5 بخش‌های فنی

جدول ۵-۲ نشان می‌دهد هر بخش فنی چه نقشی در گزارش دارد، بدون این‌که خواننده درگیر نام‌های داخلی پروژه شود.

جدول 2-5 بخش‌های فنی و نقش آن‌ها در پروژه

بخش	نقش در سامانه	اثر در گزارش
مسیریابی بک‌اند	تعریف سطح API و نسخه‌بندی مسیرها	توضیح قرارداد ارتباطی سامانه
Serializer و View	دریافت درخواست، اعتبارسنجی و پاسخ	توضیح رفتار حساب، کاتالوگ و ابزارها
Repository داده	مدیریت داده‌های کاربر، کاتالوگ، مصرف و اشتراک	توضیح مدل عملیاتی داده
رابط کاربری	ساخت صفحه‌ها و حرکت کاربر	توضیح تجربه محصول
Client فرانت‌اند	ارتباط کنترل‌شده با بک‌اند	توضیح اتصال صفحه‌ها به API
اجرای محلی	بالا آوردن فرانت‌اند و بک‌اند کنار هم	توضیح مسیر ارائه و آزمون دستی
آزمون‌ها	کنترل رفتارهای اصلی و خطاهای مهم	پشتوانه بخش ارزیابی

7-5 نمونه کد

کد زیر نمونه‌ای کوتاه از منطق پنهان کردن داده حساس را نشان می‌دهد.

```
1 SENSITIVE_KEYS = {"password", "api_key", "token", "secret", "
    authorization"}
2
3 def redact_secrets(value):
4     if isinstance(value, dict):
5         return {
6             key: "[REDACTED]" if str(key).lower() in
              SENSITIVE_KEYS else redact_secrets(item)
7             for key, item in value.items()
8         }
9     return value
```

این قطعه ساده است، اما برای چنین پروژه‌ای مهم است. سامانه با گذرواژه، API Key، توکن و اطلاعات احراز هویت سروکار دارد. اگر این داده‌ها بدون کنترل وارد پاسخ‌ها یا لاگ‌ها شوند، امنیت کاربر آسیب می‌بیند. جایگزین کردن مقدار حساس با [REDACTED] کمک می‌کند خروجی‌ها برای بررسی فنی قابل استفاده باشند، اما رازهای کاربر لو نروند.

8-5 جمع‌بندی فصل

پیاده‌سازی پروژه از چند بخش روشن ساخته شده است: فرانت‌اند برای تجربه کاربر، بک‌اند برای قرارداد API، Serializer برای کنترل ورودی و خروجی، و Repository برای کار با داده. همین ترکیب باعث می‌شود پروژه به یک پلتفرم واقعی اشتراک‌گذاری API نزدیک شود.

فصل ۶

آزمون و ارزیابی

این فصل آزمون‌های پروژه و نتیجه اجرای محلی را گزارش می‌کند. هدف آزمون این است که مطمئن شویم مسیرهای اصلی بک‌اند، رفتارهای امنیتی و قراردادهای عمومی API بعد از تغییرات ناخواسته خراب نمی‌شوند.

1-6 راهبرد آزمون

آزمون‌های بک‌اند چند بخش را پوشش می‌دهند: آماده‌بودن برنامه، پنهان‌سازی رازها، اعتبارسنجی داده، احراز هویت، کاتالوگ، مستندات، امتیازدهی، انتشار API، Caller، Studio، سازمان‌ها، اشتراک، پروفایل، طرح OpenAPI و مسیرهای متادیتای سایت. چارچوب آزمون در Django برای همین نوع کنترل رفتار سمت سرور استفاده می‌شود [۲].

راهبرد آزمون ترکیبی است. بخشی از آزمون‌ها رفتار زیرساختی را بررسی می‌کنند؛ مانند آماده‌بودن برنامه، شاخص‌ها و ترتیب برنامه‌های نصب‌شده. بخشی دیگر رفتار محصول را بررسی می‌کنند؛ مانند ثبت‌نام، ورود، مشاهده کاتالوگ، انتشار API و مدیریت اشتراک. دسته سوم آزمون‌ها به امنیت و قرارداد خروجی مربوط‌اند؛ مانند پنهان‌سازی رازها، محدودبودن مسیرهای خصوصی و معتبر بودن طرح OpenAPI.

2-6 نتایج اجرا

دستور زیر در ریشه پروژه اجرا شد:

```
python manage.py test api
```

نتیجه اجرا: 49 آزمون اجرا شد و همه با وضعیت موفق پایان یافتند. پایگاه آزمون SQLite به‌صورت خودکار ساخته و حذف شد.

موفقیت این اجرا نشان می‌دهد که وضعیت فعلی بک‌اند از نظر آزمون‌های واحد و یکپارچگی سبک، پایدار است. با این حال، نتیجه فقط برای همان مجموعه آزمون و همان محیط اجرا معتبر است. برای ادعای آمادگی تولید، باید آزمون‌های بیشتری مانند بار، امنیت، مرورگر و اجرای کامل

Docker Compose نیز به چرخه ارزیابی اضافه شود.

3-6 جدول آزمون‌ها

جدول 1-6 خلاصه آزمون‌های اجراشده

شناسه	مؤلفه	سناریو	انتظار	نتیجه واقعی	وضعیت
T1	امنیت	حذف گذرواژه، کلید و توکن از داده	رازها دیده نشوند	موفق	قبول
T2	مدل‌ها	نامک فارسی یکتا و اعتبار رنگ و امتیاز	خطا برای داده نامعتبر	موفق	قبول
T3	احراز هویت	ثبت‌نام، ورود، نشست و خروج	نشست ساخته و پاک شود	موفق	قبول
T4	کاتالوگ	فهرست، جزئیات، مستندات و endpointها	داده صفحه‌بندی	موفق	قبول
T5	حساب	پرو فایل، کلید API، سازمان و دسترسی	مسیر خصوصی با احراز هویت	موفق	قبول
T6	اشتراک	ساخت، مشاهده، لغو و تأیید پرداخت دستی	وضعیت درست checkout	موفق	قبول
T7	Studio Caller	ثبت مصرف پس از اجرا یا استقرار	رکورد مصرف ساخته شود	موفق	قبول
T8	طرح و سایت	OpenAPI، robots.txt و sitemap.xml	پاسخ معتبر	موفق	قبول

جدول نشان می‌دهد پوشش آزمون‌ها فقط به یک قابلیت محدود نیست و بیشتر مسیرهای اصلی محصول را لمس می‌کند. وجود آزمون برای مسیرهای OpenAPI و متادیتای سایت نیز اهمیت دارد، زیرا این مسیرها مستقیماً به مصرف‌کنندگان بیرونی و موتورهای جستجو مربوط می‌شوند.

آزمون‌های امنیتی نیز تضمین می‌کنند داده‌های حساس در پاسخ عادی سامانه آشکار نشوند.

4-6 محدودیت‌های آزمون

آزمون فرانت‌اند با Playwright در این اجرا دوباره انجام نشد. بنابراین نتیجه قطعی این فصل مربوط به بک‌اند است. برای کامل‌شدن ارزیابی، بهتر است جریان‌هایی مثل ورود، جستجو، مشاهده جزئیات API، Dashboard و Studio در مرورگر هم اجرا شوند [۱۲].

این محدودیت به این معنی است که رفتار تصویری و تعاملی فرانت‌اند در این گزارش دوباره اثبات نشده است. برای مثال، درست‌بودن نمایش فرم‌ها، حرکت بین صفحات، پیام‌های خطا، واکنش‌گرایی و اتصال واقعی صفحه‌ها به بک‌اند باید با اجرای تازه آزمون مرورگر یا بررسی دستی ثبت شود. بنابراین ارزیابی فعلی از نظر بک‌اند قوی‌تر از فرانت‌اند است.

5-6 جمع‌بندی فصل

آزمون بک‌اند موجود اجرا شد و همه 49 آزمون موفق بودند. برای تکمیل ارزیابی، اجرای تازه آزمون‌های فرانت‌اند، سنجش مسیر Docker Compose، افزودن آزمون بار سبک و بررسی امنیت پیکربندی پیشنهاد می‌شود.

فصل ۷

نتایج و بحث

این فصل نتیجه پروژه را با زبان روشن بیان می‌کند: چه چیزهایی آماده است، چه چیزهایی هنوز نیاز به تکمیل دارد، و وضعیت کلی سامانه برای ارائه و توسعه بعدی چگونه است.

1-7 ویژگی‌های پیاده‌سازی شده

ویژگی‌های اصلی پروژه شامل کاتالوگ عمومی، صفحه جزئیات API، مستندات، پلن‌ها، ثبت امتیاز، ثبت‌نام، ورود، نشست، خروج، پروفایل، چرخش API Key، سازمان‌ها، اشتراک، checkout دستی، مصرف، Caller، Studio و انتشار API است. این ترکیب، پروژه را به فضای API Marketplace نزدیک می‌کند [۱۱].

این ویژگی‌ها دو دسته ارزش ایجاد می‌کنند. دسته نخست برای مصرف‌کننده API است: کشف، مقایسه، مشاهده مستندات و انتخاب پلن. دسته دوم برای توسعه‌دهنده یا مالک API است: مدیریت حساب، سازمان، کلید، انتشار، مشاهده مصرف و آزمایش جریان‌ها. ترکیب این دو دسته، پروژه را از یک فهرست ساده فراتر می‌برد و به شکل یک بازارچه عملیاتی نزدیک می‌کند.

2-7 نمونه خروجی API

طرح OpenAPI برای معرفی مسیرها و پاسخ‌ها استفاده می‌شود. نمونه زیر شکل ساده‌ای از پاسخ مسیر سلامت را نشان می‌دهد [۱۰]:

```
1 GET /api/v1/system/health/
2 {
3   "status": "ok",
4   "timestamp": "2026-05-17T00:00:00Z"
5 }
```

این پاسخ ساده است، اما از نظر ارزیابی اهمیت دارد؛ زیرا نشان می‌دهد مسیر نسخه‌دار API فعال است و Client می‌تواند وضعیت سرویس را بدون ورود بررسی کند. وجود زمان در پاسخ نیز برای عیب‌یابی و مانیتورینگ اولیه مفید است.

3-7 نقاط قوت

نقاط قوت پروژه عبارت‌اند از: مسیرهای نسخه‌دار، پاسخ خطای یکنواخت، آزمون‌های متعدد، پنهان کردن رازها، پشتیبانی از فارسی، قرارداد OpenAPI و اجرای محلی با Docker Compose. از دید نگهداری، مهم‌ترین نقطه قوت پروژه جدا شدن لایه‌هاست. مسیرها، Serializerها، لایه داده و Client فرانت‌اند نقش‌های نسبتاً روشن دارند. از دید تجربه کاربر، پشتیبانی از راست‌به‌چپ و فارسی باعث می‌شود سامانه برای مخاطب ایرانی مناسب‌تر باشد. از دید توسعه، وجود آزمون‌ها و طرح OpenAPI مسیر ادامه کار و اتصال Clientهای دیگر را ساده‌تر می‌کند.

4-7 چالش‌ها و بدهی فنی

مهم‌ترین چالش، تصمیم‌نهایی درباره مرز داده و آماده‌سازی تولیدی است. تنظیمات توسعه‌ای باید از تنظیمات تولید جدا شود، رازها باید از محیط امن خوانده شوند، ورود اجتماعی به پیکربندی کامل نیاز دارد، و Caller در نسخه تولیدی باید با کنترل خطا، محدودیت زمانی و ثبت رویداد دقیق به سرویس مقصد متصل شود.

این چالش‌ها مانع نمایش پروژه نیستند، اما برای بهره‌برداری واقعی باید حل شوند. دوگانگی داده می‌تواند در آینده باعث اختلاف رفتار بین پنل مدیریت و مسیرهای API شود. تنظیمات توسعه‌ای نیز اگر بدون تغییر وارد محیط تولید شوند، خطر امنیتی ایجاد می‌کنند. همچنین پرداخت دستی و اجرای نمونه Caller باید در نسخه تولیدی با سرویس‌های واقعی، کنترل خطا، محدودیت زمانی و ثبت رویداد دقیق جایگزین یا تکمیل شوند.

5-7 بحث

پیاده‌سازی فعلی برای نمایش و آزمون قابلیت‌های اصلی بازارچه API مناسب است. با این حال، برای استفاده تولیدی باید مرز داده، محرمانگی تنظیمات، مدیریت سرویس واقعی MongoDB، آزمون‌های فرانت‌اند و مسیر پرداخت واقعی دقیق‌تر شود.

سطح بلوغ پروژه را می‌توان «نمونه کامل قابل اجرا برای محیط توسعه» دانست. قابلیت‌های اصلی محصول وجود دارند و آزمون بک‌اند از آن‌ها پشتیبانی می‌کند، اما همه نیازهای یک سامانه تولیدی هنوز کامل نشده‌اند؛ از جمله مدیریت رازها، اتصال‌های بیرونی واقعی، مشاهده‌پذیری، پایداری زیر بار و چرخه انتشار رسمی.

6-7 جمع‌بندی فصل

نتیجه پروژه یک پورتال نسبتاً کامل برای کشف، انتشار و مدیریت API است. بک‌اند از نظر آزمون‌های فعلی وضعیت خوبی دارد، اما آماده‌سازی تولیدی هنوز کار می‌خواهد. همین موضوع مبنای پیشنهاد‌های فصل پایانی است.

فصل ۸

نتیجه‌گیری و کارهای آینده

این فصل جمع‌بندی نهایی پروژه و مسیر توسعه بعدی را ارائه می‌کند. هدف این بخش، تکرار فصل‌های قبل نیست؛ بلکه می‌خواهد روشن کند پروژه امروز کجا ایستاده و قدم‌های بعدی آن چیست.

1-8 جمع‌بندی کار انجام‌شده

IranAPI Hub یک پلتفرم اشتراک‌گذاری API با بک‌اند Django REST Framework و فرانت‌اند React است. سامانه کاتالوگ، مستندات، احراز هویت، Dashboard، مصرف، اشتراک، سازمان، انتشار، Caller و Studio را پوشش می‌دهد.

در طول گزارش، پروژه از زاویه کاربر و معماری بررسی شد. فصل نیازمندی‌ها نقش‌ها و جریان‌ها را مشخص کرد. فصل معماری نشان داد این جریان‌ها چگونه بین فرانت‌اند، بک‌اند و داده تقسیم می‌شوند. فصل پیاده‌سازی رفتار بخش‌های اصلی را توضیح داد و فصل آزمون نشان داد بک‌اند با 49 آزمون موفق پشتیبانی می‌شود.

2-8 دستاوردهای اصلی

دستاوردهای اصلی پروژه شامل پشتیبانی از زبان فارسی، مسیرهای عمومی و خصوصی منظم، خروجی OpenAPI، تست‌های بک‌اند، ساختار Docker Compose و تفکیک نسبی بین لایه نمای API، سریال‌سازی، داده و رابط کاربری است.

دستاوردهای مهم دیگر، هماهنگی نسبی بین تجربه کاربر و معماری فنی است. کاربر کاتالوگ، مستندات، حساب و ابزارهای توسعه‌دهنده را می‌بیند و پشت آن، API، Serializer، لایه داده و آزمون قرار گرفته‌اند. این رابطه باعث می‌شود پروژه فقط نمایشی نباشد و از نظر فنی هم قابل ادامه‌دادن باشد.

3-8 محدودیت‌های فعلی

محدودیت‌های مستندشده عبارت‌اند از: آماده نبودن تنظیمات تولید، باقی ماندن مسیرها و مدل‌های سازگار قدیمی، نبود اجرای تازه آزمون‌های فرانت‌اند در این گزارش، نبود اثبات اتصال واقعی Caller به سرویس خارجی و نیاز ورود اجتماعی به پیکربندی ارائه‌دهنده. این محدودیت‌ها طبیعی‌اند، اما باید در برنامه ادامه کار جدی گرفته شوند. تا زمانی که تنظیمات تولید، رازها، پایگاه داده واقعی، پرداخت، اتصال بیرونی و آزمون مرورگر کامل نشده باشند، بهتر است پروژه به عنوان نسخه آموزشی یا نمونه قابل توسعه معرفی شود، نه محصول تولیدی آماده بهره‌برداری عمومی.

4-8 کارهای آینده

کارهای آینده پیشنهادی:

- انتقال رازها و تنظیمات حساس به متغیرهای محیطی و غیرفعال کردن DEBUG در تولید.
 - تصمیم‌گیری نهایی درباره مرز MongoDB و Django ORM و حذف دوگانگی غیرضروری.
 - اجرای منظم آزمون‌های Playwright و افزودن آن‌ها به فرایند CI.
 - اتصال Caller به سرویس‌های مقصد با کنترل امنیت، timeout و ثبت خطای واقعی.
 - تکمیل پرداخت واقعی، webhook و کنترل سهمیه مصرف.
 - افزودن مستندات نصب تولید، نسخه‌گذاری API و مانیتورینگ عملیاتی.
- ترتیب انجام این کارها بهتر است از ریسک‌های بنیادی شروع شود. ابتدا باید امنیت پیکربندی و رازها اصلاح شود، سپس تصمیم داده‌ای بین MongoDB واقعی و مدل‌های ORM نهایی شود. پس از آن، آزمون‌های مرورگر، پرداخت واقعی، مانیتورینگ و اتصال Caller می‌توانند با اطمینان بیشتری کامل شوند.

5-8 جمع‌بندی فصل

پروژه پایه فنی مناسبی برای یک بازارچه API دارد و آزمون‌های بک‌اند موجود آن با موفقیت اجرا شده‌اند. مسیر توسعه بعدی باید روی سخت‌سازی تولیدی، یکپارچه‌سازی داده و آزمون‌های انتهابه‌انتها متمرکز شود. با انجام این گام‌ها، سامانه می‌تواند از نمونه آموزشی کامل به محصولی پایدارتر و آماده‌تر برای استفاده واقعی تبدیل شود.

منابع

- [https:// .Python 3 Documentation .Python Software Foundation](https://docs.python.org/3/) [١]
[.docs.python.org/3/](https://docs.python.org/3/)
- [https://docs . Django Documentation .Django Software Foundation](https://docs.djangoproject.com/) [٢]
[.djangoproject.com/](https://docs.djangoproject.com/)
- [https://www . Official Documentation .Django REST Framework](https://www.djangoproject.com/rest-framework/) [٣]
[.django-rest-framework.org/](https://www.djangoproject.com/rest-framework/)
- [.https://react.dev/ .React Documentation .Meta Open Source](https://react.dev/) [٤]
- [https://www.typescriptlang . TypeScript Handbook .Microsoft](https://www.typescriptlang.org/docs/) [٥]
[.org/docs/](https://www.typescriptlang.org/docs/)
- [.https://vite.dev/guide/ .Vite Guide .Vite](https://vite.dev/guide/) [٦]
- [https://nodejs.org/ .Node.js Documentation .OpenJS Foundation](https://nodejs.org/docs/latest/api/) [٧]
[.docs/latest/api/](https://nodejs.org/docs/latest/api/)
- [https://docs.docker . Docker Compose Documentation .Docker](https://docs.docker.com/compose/) [٨]
[.com/compose/](https://docs.docker.com/compose/)
- [https://www.mongodb.com/ .MongoDB Manual .MongoDB](https://www.mongodb.com/docs/manual/) [٩]
[.docs/manual/](https://www.mongodb.com/docs/manual/)
- [https://spec . OpenAPI Specification .OpenAPI Initiative](https://spec.openapis.org/oas/latest.html) [١٠]
[.openapis.org/oas/latest.html](https://spec.openapis.org/oas/latest.html)
- [.https://docs.rapidapi.com/ .RapidAPI Docs .RapidAPI](https://docs.rapidapi.com/) [١١]
- [https://playwright.dev/ .Playwright Documentation .Microsoft](https://playwright.dev/docs/intro) [١٢]
[.docs/intro](https://playwright.dev/docs/intro)

واژه‌نامه

واژه فارسی	برابر انگلیسی
پلتفرم اشتراک گذاری API	API Marketplace
احراز هویت نشست	Session Authentication
Legacy Token	Legacy Token
Studio Flow	Studio Flow
مجوز دسترسی	Access Grant

پیوست الف

راهنمای اجرا و شواهد تکمیلی

1-الف اجرای محلی

اجرای محلی پروژه با Docker Compose انجام می‌شود:

```
1 | docker compose up --build
```

در این پیکربندی، فرانت‌اند روی `http://localhost:5173` و بک‌اند روی `http://localhost:8000/api` منتشر می‌شود.

این روش برای بررسی سریع کل سامانه مناسب است، زیرا هر دو بخش اصلی را هم‌زمان بالا می‌آورد. پس از اجرا، کاربر می‌تواند ابتدا مسیر سلامت بک‌اند را بررسی کند و سپس از طریق فرانت‌اند وارد جریان مرور کاتالوگ، ورود، داشبورد و ابزارهای توسعه‌دهنده شود. اگر یکی از سرویس‌ها اجرا نشود، باید لاگ همان سرویس در Docker Compose بررسی شود.

2-الف اجرای آزمون بک‌اند

دستور اجراشده در این گزارش:

```
1 | python manage.py test api
```

خروجی خلاصه: Ran 49 tests و OK.

این دستور فقط آزمون‌های برنامه `api` را اجرا می‌کند و برای تکرار نتیجه فصل آزمون کافی است. در صورت تغییر کد بک‌اند، اجرای دوباره این دستور باید نخستین کنترل کیفی باشد. برای تغییرات فرانت‌اند، اجرای آزمون‌های مرورگر و ساخت نسخه فرانت‌اند نیز باید به این کنترل اضافه شود.

3-الف مسیرهای مهم

جدول الف-۱ مسیرهای مهم برای بررسی دستی سامانه

مسیر	کاربرد
/api/v1/system/health/	سلامت سامانه
/api/v1/schema/openapi.json	طرح OpenAPI
/api/v1/auth/login/	ورود نشست‌محور
/api/v1/catalog/apis/	فهرست و انتشار API
/api/v1/account/usage/	تاریخچه مصرف
/api/v1/account/studio/flows/	جریان‌های Studio